

Game Theory Algorithms Problem Set

Instructor: Jing Chen

10 points each problem. Due by Lecture 10.

1. Chapter 1, Problem 3: TV schedules.

There are many other settings in which we can ask questions related to some type of “stability” principle. Here’s one, involving competition between two enterprises.

Suppose we have two television networks, whom we’ll call $\mathcal{A}$ and $\mathcal{B}$. There are n prime-time programming slots, and each network has n TV shows. Each network wants to devise a *schedule*—an assignment of each show to a distinct slot—so as to attract as much market share as possible.

Here is the way we determine how well the two networks perform relative to each other, given their schedules. Each show has a fixed rating, which is based on the number of people who watched it last year; we’ll assume that no two shows have exactly the same *rating*. A network *wins* a given time slot if the show that it schedules for the time slot has a larger rating than the show the other network schedules for that time slot. The goal of each network is to win as many time slots as possible.

Suppose in the opening week of the fall season, Network $\mathcal{A}$ reveals a schedule S and Network $\mathcal{B}$ reveals a schedule T . On the basis of this pair of schedules, each network wins certain time slots, according to the rule above. We’ll say that the pair of schedules (S, T) is *stable* if neither network can unilaterally change its own schedule and win more time slots. That is, there is no schedule S' such that Network $\mathcal{A}$ wins more slots with the pair (S', T) than it did with the pair (S, T) ; and symmetrically, there is no schedule T' such that Network $\mathcal{B}$ wins more slots with the pair (S, T') than it did with the pair (S, T) .

The analogue of Gale and Shapley’s question for this kind of stability is the following: For every set of TV shows and ratings, is there always a stable pair of schedules? Resolve this question by doing one of the following two things:

- (a) give an algorithm that, for any set of TV shows and associated ratings, produces a stable pair of schedules; or
- (b) give an example of a set of TV shows and associated ratings for which there is no stable pair of schedules.

2. Chapter 1, Problem 4: Extension of the Gale-Shapley algorithm.

Gale and Shapley published their paper on the Stable Matching Problem in 1962; but a version of their algorithm had already been in use for ten years by the National Resident Matching Program, for the problem of assigning medical residents to hospitals.

Basically, the situation was the following. There were m hospitals, each with a certain number of available positions for hiring residents. There were n medical students graduating in a given year, each interested in joining one of the hospitals. Each hospital had a ranking of the students in order of preference, and each student had a ranking of the hospitals in order of preference. We will assume that there were more students graduating than there were slots available in the m hospitals.

The interest, naturally, was in finding a way of assigning each student to at most one hospital, in such a way that all available positions in all hospitals were filled. (Since we are assuming a surplus of students, there would be some students who do not get assigned to any hospital.)

We say that an assignment of students to hospitals is *stable* if neither of the following situations arises.

- First type of instability: There are students s and s' , and a hospital h , so that
 - s is assigned to h , and
 - s' is assigned to no hospital, and
 - h prefers s' to s .
- Second type of instability: There are students s and s' , and hospitals h and h' , so that
 - s is assigned to h , and
 - s' is assigned to h' , and
 - h prefers s' to s , and
 - s' prefers h to h' .

So we basically have the Stable Matching Problem, except that (i) hospitals generally want more than one resident, and (ii) there is a surplus of medical students.

Show that there is always a stable assignment of students to hospitals, and give an algorithm to find one.

3. Chapter 1, Problem 6: Ship maintenance.

Peripatetic Shipping Lines, Inc., is a shipping company that owns n ships and provides service to n ports. Each of its ships has a *schedule* that says, for each day of the month, which of the ports it's currently visiting, or whether it's out at sea. (You can assume the

“month” here has m days, for some $m > n$.) Each ship visits each port for exactly one day during the month. For safety reasons, PSL Inc. has the following strict requirement:

(†) No two ships can be in the same port on the same day.

The company wants to perform maintenance on all the ships this month, via the following scheme. They want to *truncate* each ship’s schedule: for each ship S_i , there will be some day when it arrives in its scheduled port and simply remains there for the rest of the month (for maintenance). This means that S_i will not visit the remaining ports on its schedule (if any) that month, but this is okay. So the *truncation* of S_i ’s schedule will simply consist of its original schedule up to a certain specified day on which it is in a port P ; the remainder of the truncated schedule simply has it remain in port P .

Now the company’s question to you is the following: Given the schedule for each ship, find a truncation of each so that condition (†) continues to hold: no two ships are ever in the same port on the same day.

Show that such a set of truncations can always be found, and give an algorithm to find them.

Example. Suppose we have two ships and two ports, and the “month” has four days. Suppose the first ship’s schedule is

port P_1 ; at sea; port P_2 ; at sea

and the second ship’s schedule is

at sea; port P_1 ; at sea; port P_2

Then the (only) way to choose truncations would be to have the first ship remain in port P_2 starting on day 3, and have the second ship remain in port P_1 starting on day 2.

4. Chapter 2, Problem 1: Worst Case.

Show that for any deterministic evaluation algorithm, there is an instance of $T_{d,k}$ that forces the algorithm to read the values on all d^{2k} leaves.

5. Chapter 2, Problem 3: Evaluation Problem.

Consider a uniform rooted tree of height h —every leaf is at distance h from the root. The root, as well as any internal node, has three children. Each leaf has a Boolean value associated with it. Each internal node returns the value returned by the majority of its children. The evaluation problem consists of determining the value of the root; at each step, an algorithm can choose one leaf whose value it wishes to read.

- (a) Show that for any deterministic algorithm, there is an instance (a set of Boolean values for the leaves) that forces it to read all $n = 3^h$ leaves.
- (b) Consider the recursive randomized algorithm that evaluates two sub-trees of the root chosen at random. If the values returned disagree, it proceeds to evaluate the third sub-tree. Show that the expected number of leaves read by this algorithm (on any instance) is at most $n^{0.9}$.

6. Chapter 2, Problem 5: Minimax.

Let (a_{ij}) be a $m \times n$ matrix, let the vector $(p_1, p_2, \dots, p_m)$ consist of reals in $[0,1]$ such that $\sum_{i=1}^m p_i = 1$, and let $(q_1, q_2, \dots, q_n)$ consist of reals in $[0,1]$ such that $\sum_{j=1}^n q_j = 1$. Prove algebraically that $\max_q \min_i \sum_{j=1}^n a_{ij} q_j \leq \min_p \max_j \sum_{i=1}^m p_i a_{ij}$.

7. Chapter 2, Problem 7: Search.

(Due to A.M. Karp.) You are given an array A containing n numbers in sorted order. In one step, an algorithm may specify an integer $i \in [1, n)$, and is given the value of $A[i]$ in return. Determine lower and upper bounds on the expected number of steps taken by a Las Vegas randomized algorithm to determine whether or not a given key k is present in the array.

8. Chapter 2, Problem 8: Perfect Match.

(Due to A.M. Karp.) In a graph with n vertices, where n is even, a perfect matching is a set of $n/2$ edges, no two of which meet at a common vertex. Consider a randomized algorithm that takes an n -vertex graph as input and correctly determines whether the graph has a perfect matching. At each step the algorithm asks a question of the form “Is there an edge between vertex i and vertex j ?” The complexity of the algorithm is defined as the maximum, over all n -vertex graphs G , of the expected number of questions $C(n)$ asked when the input graph is G . Prove: $C(n) = \Omega(n^2)$.

9. Chapter 19, Problem 7: $\alpha > n^2$.

Show that when $\alpha > n^2$ all Nash equilibria of the local connection game are trees and the price of anarchy is bounded by a constant.

10. Chapter 19, Problem 9: PoA.

Prove that in the global connection game, the price of anarchy can never exceed k , the number of players.

11. Chapter 19, Problem 10: Potential Function.

Consider the following weighted generalization of the global connection game. For each player i , we have a weight $w_i > 0$. As before, each player selects a single path connecting her source and sink. But instead of sharing edge costs equally, players are now assigned cost shares in proportion to their weight. In particular, for a strategy vector S and edge e , let S_e denote those players whose path contains e , and let $W_e = \sum_{i \in S_e} w_i$ be the total weight of these players. Then player i pays $c_e w_i / W_e$ for each edge $e \in P_i$. Note that if all players have the same weight, this is the original game. Show that, in general, this game does not have an exact potential function.

12. Chapter 19, Problem 13: Ordinal Potential Game.

Suppose that we have a game G and a function $\Phi(S)$ mapping game states to reals with the following property: for any strategy vectors $S = (S_1, S_2, \dots, S_k)$, and any alternate strategy $S'_i \neq S_i$ for some player i , then if $S' = (S_{-i}, S'_i)$, we have that $\Phi(S) - \Phi(S')$ and $u_i(S') - u_i(S)$ share the same sign. Thus $\Phi(S)$ behaves like an exact potential function, except instead of tracking a player's improvement exactly, it simply tracks the direction of the improvement; when a player makes an improving move, the potential function decreases. We call such a function an *ordinal potential function*, and G an *ordinal potential game*.

(a) Prove that if G is an ordinal potential game, then best response dynamics always converge to a Nash equilibrium.

(b) Prove that the converse is also true; if, from any starting configuration, best response dynamics always converge to an equilibrium, then G is an ordinal potential game.